

如何优化机器人移动行为

English version: [🌐 How to Optimize Robot Movement Behavior](#)

<https://www.bilibili.com/video/BV1JtMx6bEwu>

移动行为策略

当前的移动行为逻辑建立在**反应式控制**的基础上，每帧根据当前环境**即时反应**，充分适配本次比赛场地小、障碍少、实时性高的场景。针对双足机器人的步态特点，移动速度由正方向速度 `vx` 和角度 `vyaw` 控制，不调整水平方向速度 `vy`。

确定 `移动目标` 后，策略从以下三个层面规划具体的 `移动行为`：

代码块

```
1  战术目标点（场系坐标）
2      |
3      ▼
4  第一层：路径绕行（_avoidance_target）
5      沿直线检测障碍物，有障碍 → 算绕行 via 点
6      修改目标点，不修改控制指令
7      |
8      ▼
9  第二层：行走控制（_compute_velocity）
10     Unicycle 模型：目标点 → vx + vyaw
11     三种模式：停止 / 纯转向 / 前进+跟随转向
12     |
13     ▼
14  第三层：转向避让（_apply_yaw_avoidance）
15     检测近邻，叠加 vyaw 偏置"偏开"
16     不修改目标点，不修改 vx
17     |
18     ▼
19  MoveIntent(vx, vy=0, vyaw)
```

三层策略各有侧重，第一层考虑障碍物，计算绕行点，设定新的移动目标；第二层计算走多快和怎么转；第三层考虑障碍物影响，为影响较大的障碍物额外增加角速度。三层计算的结果综合到一起，生成移动意图 `MoveIntent`。`MoveIntent` 最终交由执行层下发机器人具体移动命令。

移动策略在 `tactics/motion.py` 中 `MotionController.move_to_target` 中实现。接下来，我们结合代码逐层理解移动行为策略。

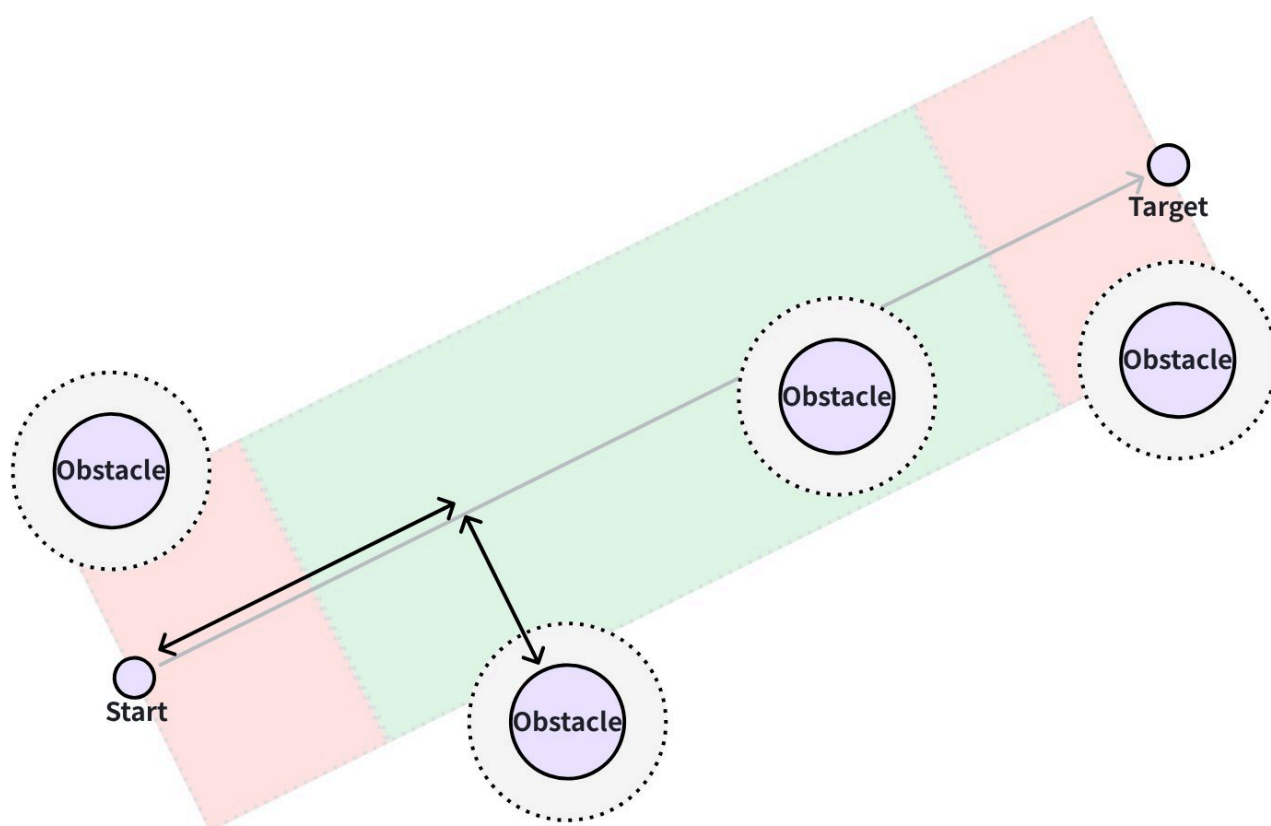
第一层 路径绕行 解析

路径绕行是三层架构的第一层，负责解决**障碍挡路**的问题。

绕行的计算逻辑是：从当前位置画一条直线到目标，如果中途有障碍物落在路径走廊里，就在障碍侧面生成一个绕行 via 点，把它当作新目标。这样底层行走控制自然会绕过去——不需要改速度算法。

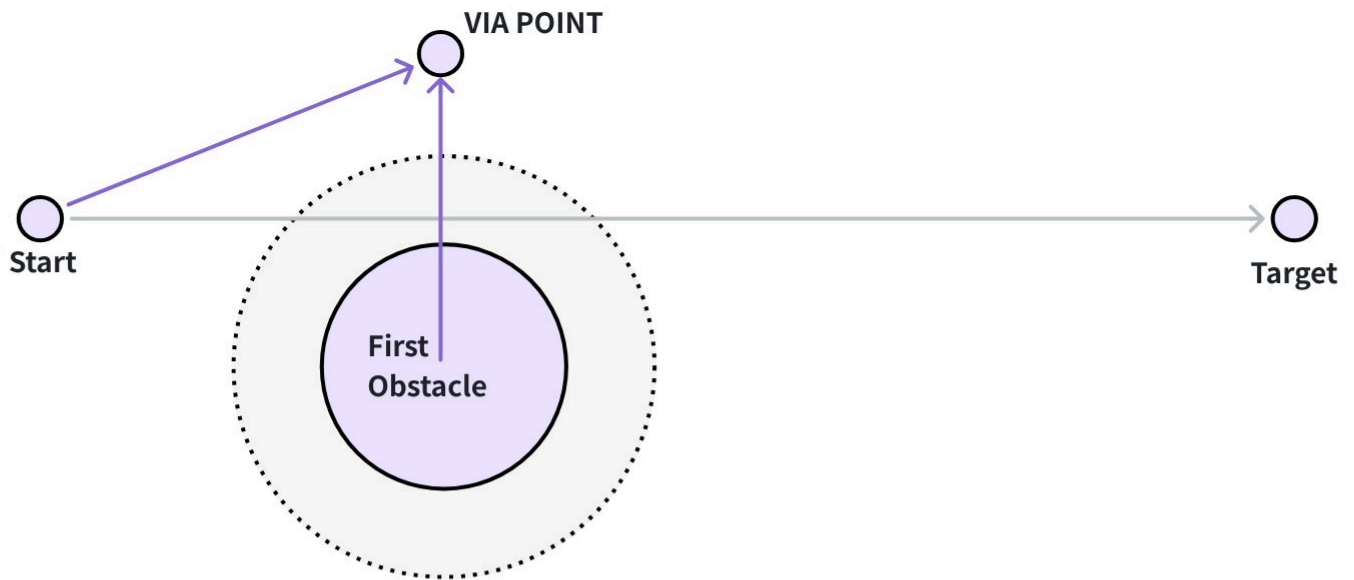
实际场景中，障碍物指同在场上的其他球员或球门。障碍物可以离我很远，但只要它挡在路径中段就应当考虑；障碍物也可以离我很近，但若不在路径走廊里就不考虑。

首先，我们需要找到最近的障碍物，此逻辑在 `_first_blocking_obstacle` 中实现。这个函数能够根据移动到目标的起点和终点信息，结合 `PlayContext`，计算出需要最先考虑的障碍物。



如果没有需要考虑的障碍物，`_first_blocking_obstacle` 将返回 `None`。即不增加绕行点，保持原目标点。

如果有障碍物，则需要使用 `_choose_avoid_side` 判断绕行方向：障碍物在路径左侧，向右绕行，障碍物在路径右侧，向左绕行。使用 `_via_pose` 计算出绕行点。



注意：_via_pose方法返回Pose2D类型的位姿信息，而不仅是绕行坐标。位姿中的方向指向原目标点，这样，我们可以在接下来两层的速度计算中，即让机器人移动到指定坐标，又面向最终目标点。

最终，如果生成的绕行点在场地外，调整到场地内部。

总结一下第一层避障的逻辑：

计算逻辑示意

```

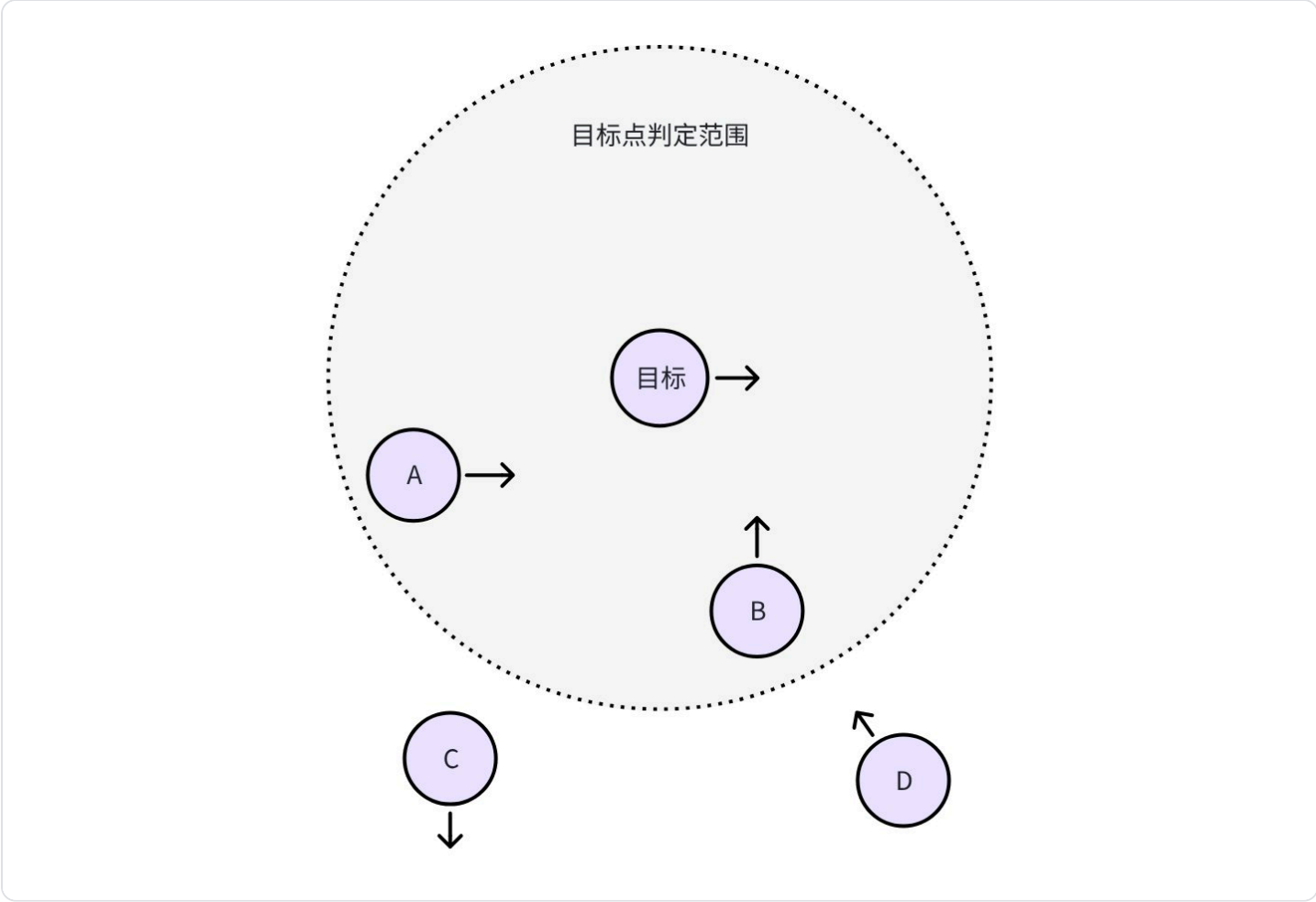
1  _avoidance_target(player_id, start, target, context)
2  |
3  |—— _first_blocking_obstacle(start → target)
4  |    沿路径投影，找第一个挡路的障碍
5  |    along 落在路径中段 且 lateral < 障碍半径+safety_margin
6  |    返回 along 最小（离起点最近）的那个
7  |
8  |—— 若无障碍 → 清除 avoid_side_by_player 记忆，返回原 target
9  |
10 |—— 若有障碍：
11 |    |—— _choose_avoid_side(start, target, obstacle) [仅首次]
12 |    |    障碍在路径左侧 → 从右绕(-1)；障碍在右侧 → 从左绕(+1)
13 |    |    结果存入 _avoid_side_by_player[player_id]，跨帧记忆
14 |    |
15 |    |—— _via_pose(start, target, obstacle, side_sign)
16 |    |    1. 投影障碍到路径上的最近接触点 closest
17 |    |    2. 沿左侧法线偏移 obstacle.radius + safety_margin
18 |    |    3. via 朝向指向原 target
19 |    |
20 |—— TeamFieldFrame.clamp_inside_field(via) → 裁回场内，返回

```

最经过第一层的计算，我们确定了更合理的移动目标。

第二层 行走控制 解析

行走控制主要负责计算出球员运动的线速度 `vx` 和角速度 `vyaw`，在 `tactics/motion.py` 中的 `MotionController._compute_velocity` 实现。



第二层的行走控制逻辑如图所示：对于A，到达目标点判定范围且和停止目标方向基本一致，可以停止；对于B，到达目标点判定范围，但朝向不合要求，在原地转向；对于C，未到达目标点，且没有朝向目标点，先纯转向，直到面向目标点；对于D，未到达目标点，但朝向目标点，前进到目标点。

需要注意的是，球员会持续关注自己的位姿状态。随着球员旋转和前进，一个处于C状态的球员通常会依次经过D、B、A三个状态，最终完成移动。

在D成为B的运动过程中，`MotionController` 额外提供了两个有用的方法，`_linear_speed`，`_angular_velocity` 分别计算线速度和角速度。两个方法均设计了动态微调机制，和目标越远，速度/角速度越大。

我们可以把控制逻辑整理成下面的表述：

1. 如果已经到达目标点且朝向正确方向，停止。
2. 如果到达目标点但朝向不正确，纯转向。
3. 如果未到达目标点且方向误差大，先转向

4. 如果未到达目标点，边前进边转向。

经过第二层的计算，我们得到了新的指令，包含新目标、线速度、角速度信息。

第三层 转向避让 解析

这一层解决障碍物较近，需要避让的问题。解决方式是在第二层计算出的角速度`vyaw`上增加额外偏置。

比如，当前命令是前进 ($v_x > 0$)，身边有队友或对手太近。由于双足底盘不能横移避开，于是让机器人多转一点，沿"绕开近邻的新方向"前进。

转向避让由 `_apply_yaw_avoidance` 实现，可以看做通过进一步分析 `PlayContext` 后，对第二层计算结果的修正。

三层计算在 `move_to_target` 中串联，生成最终的命令。

策略效果演示

你可以尝试用下面不涉及避障的建议控制算法代替 `move_to_target`。你将看到机器人将无视障碍，以非常“耿直”的方式移动。

代码块

```
1  def move_to_target(  
2      self,  
3      player_id: int,  
4      context: PlayContext,  
5      target: Pose2D,  
6      reason: str,  
7      arrive_distance: float | None = None,  
8      hold_vyaw: float = 0.0,  
9      avoid_opponents: bool = False,  
10 ) -> RobotCommand:  
11  
12     robot = context.teammates.get(player_id)  
13     if robot is None or robot.pose is None:  
14         return RobotCommand.stop(f"{reason}: waiting for pose")  
15  
16     return self._compute_velocity(  
17         robot.pose,  
18         target,  
19         "",  
20         arrive_distance,  
21         hold_vyaw,  
22     )
```

附录：相关配置参数速查

参数可在 `src/soccer_framework/config.py` 中修改。

类型	参数	默认值	说明
速度上限	max_linear_speed	0.8 m/s	线速度硬上限，motion 层 clamp
速度上限	max_angular_speed	1.0 rad/s	角速度硬上限，motion 层 clamp
行走控制	_ARRIVE_DISTANCE	0.15m	距目标 15cm 内认为到达
行走控制	_ARRIVE_ANGLE	0.20rad (~11.5°)	朝向误差 11.5° 内认为对齐
行走控制	_TURN_THRESHOLD	0.5rad (~28.6°)	大于此值纯转向，不大于则前进 + 转向
行走控制	_ANGULAR_SPEED_FLOOR	0.25 rad/s	角速度最低保底（死区外）
行走控制	_ANGULAR_DEAD_ZONE	0.15rad (~8.6°)	小于此值纯比例控制，不施 floor
行走控制	_LINEAR_SPEED_FLOOR	0.3 m/s	线速度最低保底
行走控制	_LINEAR_GAIN	0.9	$v_x = 0.9 \times \text{distance} \times \cos(\text{error})$
路径绕行	opponent_obstacle_radius	0.55m	对手障碍半径（直径 1.1m）
路径绕行	teammate_obstacle_radius	0.48m	队友障碍半径（直径 0.96m）
路径绕行	obstacle_safety_margin	0.22m	障碍半径之外的额外安全余量
路径绕行	obstacle_start_ignore_distance	0.35m	离起点这么近的障碍忽略（贴身不抖）
路径绕行	obstacle_target_ignore_distance	0.35m	离目标这么近的障碍忽略（到位不卡）
转向避让	yaw_avoid_horizon_sec	1.0s	向前预测近邻轨迹的时长
转向避让	yaw_avoid_min_distance_m	0.78m	当前 / 预测距离 < 此值才施加偏置
转向避让	yaw_avoid_bias_max	0.6 rad/s	单个近邻产生的最大 vyaw 偏置（按 scale 衰减）